

Deep Research: Avatrada 57 Topic 050

Engineering Report: Tailscale VPN Overlay for Secure Cloud Infrastructure

TO: Engineering Lead **FROM:** Autonomous Technical Researcher **DATE:** October 26, 2023 **SUBJECT:** Deep-Dive Analysis of Tailscale for Secure Cloud Trading Server Access

1.0 Executive Summary

This report provides a rigorous engineering analysis of using Tailscale, a WireGuard®-based VPN overlay network, to establish a zero-trust security posture for a cloud-hosted trading server on Google Cloud Platform (GCP). The primary objective, as outlined in the source prompt, is to create a "secure access network" with no open public ports, where all access is mandated through the encrypted Tailscale overlay.

The analysis confirms that Tailscale is an exceptionally well-suited solution for this use case. It allows for the complete removal of the public internet attack surface (e.g., ports 22, 80, 443) by shifting the access control paradigm from network-level firewalls to identity-aware, software-defined policies (ACLs). This report deconstructs Tailscale's architecture, provides a detailed implementation strategy for the specified GCP environment, and performs a critical analysis of potential failure modes and optimization opportunities.

2.0 Technical Deconstruction

Tailscale creates a flat, private, and encrypted mesh network between authenticated devices. Its architecture is best understood by breaking it down into its core components:

2.1 Core Architecture

1. **Control Plane (Coordination Server):** This is Tailscale's centralized "brain," hosted by Tailscale Inc. It is responsible for:

- **Authentication:** Integrating with an Identity Provider (IdP) like Google, Microsoft, Okta, etc., to verify user and device identity.
- **Public Key Exchange:** Securely distributing the public keys of all nodes within a "tailnet" to each other. It uses a sophisticated key exchange mechanism but never handles the private keys, which remain on the devices.
- **Network Map Distribution:** Telling each node about the other nodes it is allowed to connect to, based on the Access Control Lists (ACLs).
- **The control plane does *not* handle any of the actual network traffic (data plane).**

2. **Data Plane (WireGuard® Tunnels):** The actual data traffic flows through peer-to-peer, encrypted tunnels.

- **Protocol:** Tailscale uses WireGuard® for its data plane. WireGuard is a modern, high-performance VPN protocol known for its cryptographic soundness (using ChaCha20, Poly1305, etc.) and small codebase, making it highly secure and efficient.
- **Peer-to-Peer Mesh:** Whenever possible, Tailscale nodes establish a direct, end-to-end encrypted connection. This minimizes latency and avoids routing traffic through a central gateway.

3. **NAT Traversal (STUN/ICE):** To establish direct peer-to-peer connections, nodes behind different firewalls and NATs must discover each other's public IP addresses and open paths through the firewalls. Tailscale uses standard protocols like STUN and ICE to facilitate this "hole punching."

4. **DERP Relays (Designated Encrypted Relay for Packets):** In cases where NAT traversal fails (e.g., due to highly restrictive symmetric NATs), traffic is relayed through Tailscale's global network of DERP servers. The connection remains end-to-end encrypted; the DERP server cannot decrypt the traffic. It simply forwards encrypted packets from one node to another.

This ensures connectivity even in the most challenging network environments.

5. **tailscale0 Virtual Network Interface:** On each device, the Tailscale client creates a virtual network interface (e.g., `tailscale0` on Linux). This interface is assigned a stable, private IP address from the `100.64.0.0/10` CGNAT address space. All traffic destined for other nodes in the tailnet is routed through this interface, where it is transparently encrypted and sent over the WireGuard tunnel.

3.0 Implementation Strategy

This section provides a step-by-step guide to implement the secure trading server environment as specified in the prompt.

3.1 Prerequisites

- A Google Cloud Platform (GCP) project.
- A Tailscale account, connected to your preferred Identity Provider.
- A local machine (e.g., a developer's laptop) with Tailscale installed.
- A GCP VM instance for the "cloud trading server."

3.2 Step 1: Configure GCP Firewall to Block All Ingress

The first step is to lock down the server's network interface at the cloud provider level. We will create a high-priority firewall rule that denies all incoming traffic from any source.

```
# Define variables for clarity
GCP_PROJECT="your-gcp-project-id"
NETWORK_NAME="default" # Or your custom VPC network name
RULE_NAME="deny-all-public-ingress"

# Create a firewall rule with a high priority (lower number = higher priority)
# to deny all ingress traffic on all protocols and ports.
gcloud compute firewall-rules create ${RULE_NAME} \
```

```
--project=${GCP_PROJECT} \  
--network=${NETWORK_NAME} \  
--action=DENY \  
--direction=INGRESS \  
--rules=all \  
--source-ranges="0.0.0.0/0" \  
--priority=1000 \  
--description="Block all public ingress traffic to enforce Tailscale-only  
access."
```

Rationale: This rule effectively makes the VM invisible to the public internet. Standard GCP firewall rules that allow SSH (port 22) typically have a priority of 65534. Our rule with priority 1000 will be evaluated first and will deny the connection, rendering the default allow rules moot.

3.3 Step 2: Install and Configure Tailscale on the Server

With the firewall in place, access the server via the GCP Console's serial console or an IAP (Identity-Aware Proxy) tunnel for this one-time setup.

```
# Run the official Tailscale installation script  
curl -fsSL https://tailscale.com/install.sh | sh  
  
# Start Tailscale and connect it to your tailnet  
# --ssh enables Tailscale SSH for enhanced security (recommended)  
# --accept-routes enables access to subnets if configured later  
sudo tailscale up --ssh --accept-routes
```

Follow the URL provided by the command to authenticate the server and add it to your tailnet.

3.4 Step 3: Configure Tailscale ACLs for Device-Specific Access

The prompt asks to restrict access to a "specific device key." Tailscale ACLs operate on higher-level abstractions like **users**, **groups**, and **tags**, not raw

device keys. The correct and scalable approach is to use a tag for the server and grant access to a specific user or group.

1. **Tag the Server:** In the Tailscale Admin Console, find your new trading server. Click the three-dot menu and select "Edit machine tags...". Apply a tag, for example, `tag:trading-server`.
2. **Define the ACL Policy:** Go to the "Access Controls" page in the Admin Console. Use the following policy. This policy is written in HuJSON.

```
{
  // Define who can apply tags to machines.
  // Restrict this to administrators for security.
  "tagOwners": {
    "tag:trading-server": ["group:admin"],
  },

  // Define groups of users based on your IdP.
  "groups": {
    "group:traders": ["user1@example.com", "user2@example.com"],
    "group:admin": ["admin@example.com"],
  },

  // Define access rules. The default policy is to deny all traffic.
  // Rules are evaluated in order. The first match wins.
  "acIs": [
    {
      // Allow users in the 'traders' group to SSH into the trading server.
      "action": "accept",
      "src": ["group:traders"],
      "dst": ["tag:trading-server:22"], // Port 22 on any machine with this
tag
    },
    {
      // Allow admins to access any port for maintenance.
      "action": "accept",
      "src": ["group:admin"],
      "dst": ["tag:trading-server:*"],
    },
  ],
}
```

```
// Enable Tailscale SSH, which uses ACLs for authorization
// instead of relying on sshd_config and authorized_keys.
"ssh": [
  {
    "action": "check", // 'check' mode runs SSH as the user, but checks ACLs
    first.
    "src": ["group:traders", "group:admin"],
    "dst": ["tag:trading-server"],
    "users": ["autogroup:nonroot", "ubuntu", "ec2-user"], // Allowed OS
    usernames
  },
],
}
```

Explanation of the ACL Logic:

- `tagOwners` : A security control that specifies only admins can assign the `tag:trading-server` tag, preventing privilege escalation.
- `groups` : Defines reusable aliases for users from your identity provider.
- `acls` : The core firewall rules. We explicitly `accept` traffic from the `group:traders` to port 22 on the `tag:trading-server`. All other traffic is implicitly denied by default.
- `ssh` : This section configures **Tailscale SSH**. When a user from `group:traders` runs `tailscale ssh ubuntu@trading-server`, Tailscale authenticates them, checks this policy, and establishes the SSH session without ever needing a private key on the client machine.

3.5 Step 4: Access the Server

From your local machine (which must also be logged into Tailscale), you can now access the server.

```
# Find the server's Tailscale IP or use its machine name (MagicDNS)
tailscale status

# Access via standard SSH over the Tailscale interface
```

```
ssh your-user@trading-server-name.your-tailnet.ts.net
```

```
# OR, preferably, use Tailscale SSH for keyless authentication  
tailscale ssh your-user@trading-server-name
```

Any attempt to connect to the server's public IP address will be blocked by the GCP firewall and time out.

4.0 Critical Analysis

4.1 Potential Failure Modes

1. **Control Plane Unavailability:** If Tailscale's coordination servers go offline, the data plane is unaffected. Existing connections will remain active. However, new nodes cannot join the network, and ACL changes will not propagate until the control plane is restored. For a trading system, this means a new trader could not get access during an outage, but existing sessions would work.
2. **ACL Misconfiguration:** A syntax error or logical mistake in the ACL policy could inadvertently lock all users, including administrators, out of the server. **Mitigation:** Always use the "Test Rules" feature in the ACL editor and have a secondary out-of-band access method (like GCP Serial Console) for emergencies.
3. **Identity Provider (IdP) Outage:** If your IdP (e.g., Google Workspace) is down, new users cannot authenticate to Tailscale. Existing sessions, governed by key expiry, will continue to function. **Mitigation:** Tailscale's key expiry is typically long (e.g., 30 days), but for critical systems, consider the impact of a prolonged IdP outage.
4. **Key Compromise:** If a trader's laptop is compromised, the attacker gains access to the tailnet with that user's permissions. **Mitigation:** Immediately revoke the compromised device from the Tailscale Admin Console. Enforce short session timeouts via ACLs and use device posture features (e.g., require screen lock) if available on your plan.

4.2 Edge Cases

1. **Ephemeral or Auto-Scaling Servers:** If trading servers are created and destroyed frequently, manual tagging is not feasible. **Solution:** Use Tailscale **Auth Keys**. These are pre-authorized keys that can be passed to a new VM via user data. The key can be configured to automatically tag the new server (e.g., `tailscale up --authkey=tskey-auth-xyz --hostname=trading-bot-` and the auth key is configured to add `tag:trading-server`).
2. **Need for Egress to Public Internet:** The trading server may need to connect to external APIs (e.g., market data feeds). The GCP firewall rule we created does not block egress. However, for a fully locked-down environment, you can configure an **Exit Node** on Tailscale. All public internet traffic from the trading server would then be forced through a specific, audited node in your tailnet before reaching the internet.
3. **Accessing other GCP Services:** The server might need to access a Cloud SQL database or other services within the same VPC that are not on the tailnet. **Solution:** Configure the Tailscale client on the server as a **Subnet Router**. This allows other nodes in the tailnet (e.g., a trader's laptop) to access the entire GCP VPC subnet (e.g., `10.1.2.0/24`) through the encrypted tunnel to the trading server.

4.3 Optimizations

1. **Self-Host the Control Plane (Headscale):** For maximum security and to eliminate reliance on a third-party service, organizations can self-host the open-source Tailscale control plane, **Headscale**. This is a significant operational undertaking but provides complete data sovereignty.
2. **Leverage Tailscale SSH Exclusively:** Transition fully to `tailscale ssh` . This eliminates the need to manage `authorized_keys` files on servers, provides centralized audit logs of SSH sessions, and allows for session recording. It is a significant security and operational improvement over traditional SSH.
3. **Micro-segmentation with ACLs:** The provided ACL is simple. A more advanced setup would involve multiple tags for different services (e.g.,

`tag:database` , `tag:app-server`) and highly specific rules allowing traffic only between necessary components on specific ports, achieving true zero-trust micro-segmentation within your application stack.